

Informe — Agente RAG Asistente DNI · Valencia

Inteligencia Artificial · Grado en Tecnologías Interactivas · UPV
Denys Litvynov · Alan Guevara · Juan Bautista Peris
Valencia · mayo de 2026

Asistente conversacional sobre la asociación Damos Nuestra Ilusión (Valencia) basado en recuperación aumentada por generación (RAG) sobre un corpus de 16 documentos.

1. Arquitectura elegida y justificación

Se ha optado por arquitectura single-agent. La decisión se basa en la escala del problema: el corpus DNI tiene 16 documentos y ~92 KB de texto, lo que no justifica la complejidad operativa de una arquitectura hexagonal. El sistema single-agent permite iterar rápido sobre la calidad del retrieval y del prompt, que son los dos factores que más impactan la calidad de las respuestas en un corpus pequeño.

La arquitectura single-agent no significa un sistema monolítico. El código está organizado en módulos con responsabilidades claras: chunker.py (troceo), embedder.py (vectorización), retriever.py (indexación y búsqueda), generator.py (llamada al LLM), prompts.py (construcción del prompt) y pipeline.py (orquestración). El punto de entrada consultar.py actúa como adaptador fino que delega en pipeline.py, cumpliendo el contrato de interfaz del enunciado (opción A, módulo Python).

2. Decisiones tomadas a lo largo del desarrollo

Decisión 1 — Chunking adaptativo según formato del documento

Los 16 archivos del corpus tienen dos formatos distintos: narrativo (prosa larga) y FAQ (pares Q:/A:). Si el splitter estándar corta entre la pregunta y su respuesta en un archivo FAQ, el chunk resultante pierde contexto semántico y el retrieval se degrada. Se implementó en chunker.py una detección automática del formato: si un documento tiene 3 o más líneas que comienzan con "Q:", se procesa agrupando cada par Q/A completo antes de pasar por RecursiveCharacterTextSplitter. Los documentos narrativos usan el splitter estándar con chunk_size=500 y chunk_overlap=100. Además se filtra cualquier chunk con menos de 80 caracteres, eliminando títulos de sección sueltos que el splitter generaba y que contaminaban el retrieval con chunks sin contenido informativo.

Decisión 2 — ChromaDB persistente en disco con métrica coseno

Se usa ChromaDB con PersistentClient en data/chroma/, de modo que el índice se construye una sola vez y las consultas posteriores no requieren re-indexar. La colección se crea con hnsw:space=cosine, que produce scores acotados en [-1, 1] e interpretables directamente como similitud semántica. Con la métrica L2 por defecto de ChromaDB los scores tienen magnitudes no comparables entre queries.

Decisión 3 — Prompt anti-alucinación con detección de contradicciones

El corpus contiene contradicciones reales entre documentos (por ejemplo, 01_faq_dni.txt indica que los desayunos son a las 8:00 y 11_horarios_ubicaciones.txt indica entre las 9:00 y las 12:00). El prompt instruye explícitamente al modelo a presentar ambas versiones cuando detecta información contradictoria, citando el archivo fuente de cada una, en lugar de elegir una o inventar una tercera versión. El prompt también incluye la instrucción de responder exactamente "No tengo esa información en mis fuentes" ante preguntas fuera del corpus, sin añadir nada más.

Decisión 4 — Modelo de embeddings nomic-embed-text

Se eligió nomic-embed-text por tres razones: es multilingüe (funciona bien en español), tiene 768 dimensiones (buena capacidad semántica sin coste computacional excesivo), pesa 274 MB (descargable en minutos), y no requiere red externa al ejecutarse vía Ollama local.

Decisión 5 — Carga del .env antes de los imports del paquete

La clase Settings en config.py se inicializa en tiempo de import mediante SETTINGS = Settings.from_env(). Esto significa que cualquier llamada a load_dotenv() posterior al import no tiene efecto sobre SETTINGS. Se resolvió cargando el .env manualmente con os.environ.setdefault() en los scripts de entrada (build_index.py y benchmark.py) antes de cualquier import del paquete agente_rag.

3. Flujo detallado de una consulta

Cuando el corrector ejecuta *from consultar import consultar* y llama a consultar("¿Qué es DNI?"), ocurre lo siguiente en orden:

1. consultar.py recibe la pregunta y llama a answer() de pipeline.py.
2. pipeline.py llama a retrieve(question, k=5) de retriever.py.
3. retriever.py abre la colección ChromaDB persistente en data/chroma/. Llama a embed(question) de embedder.py, que hace POST a http://localhost:11434/api/embed con el modelo nomic-embed-text y devuelve un vector de 768 dimensiones. ChromaDB calcula la similitud coseno entre ese vector y los 272 vectores almacenados y devuelve los 5 chunks con mayor similitud (top-k=5), junto con sus metadatos (nombre del archivo fuente) y sus scores.
4. pipeline.py llama a build_prompt(question, retrieved_chunks) de prompts.py. El prompt incluye el rol del asistente, las reglas anti-alucinación, el contexto formado por los 5 chunks con su archivo fuente indicado, y la pregunta.
5. pipeline.py llama a generate(prompt) de generator.py, que hace POST a http://localhost:11434/api/generate con el modelo configurado (por defecto qwen2.5:3b) y captura la respuesta junto con las métricas: prompt_eval_count (tokens de entrada), eval_count (tokens de salida), eval_duration (nanosegundos de generación) y el tiempo de pared total.
6. pipeline.py ensambla el diccionario de respuesta con los campos: respuesta (texto generado), fuentes (lista de archivos fuente sin duplicados, en orden de aparición), chunks (lista de objetos con source, text y score), metricas (prompt_tokens, output_tokens, tokens_per_sec, latencia_s, modelo) y trazas (None).

7. consultar.py devuelve ese diccionario al corrector.

El tiempo total en CPU pura con qwen2.5:3b es de entre 15 y 45 segundos dependiendo de la longitud de la respuesta. Con gemma3:27b en PoliGPT el tiempo baja a 1-3 segundos.

4. Resultados sobre las preguntas de prueba

Se probaron 12 preguntas cubriendo todos los tipos del Anexo A del enunciado. El pipeline se mantuvo constante (chunking Q/A + ChromaDB coseno + nomic-embed-text + k=5) variando únicamente el modelo de generación.

ID	Pregunta	qwen2.5:3b	llama3.2:3b	gemma3:27b	llama3.3:70b
q01	¿Qué es DNI?	✓	✗	✓	✓
q02	¿A qué hora son los desayunos?	✓	✓	✓	✓
q03	¿Cómo me apunto a desayunos?	✓	✓	✓	✓
q04	¿En qué se diferencian RESIS y COLES?	✗	✗	✗	✗
q05	¿Cuántos voluntarios tiene DNI?	✓	✓	✓	✓
q06	¿Dónde es el punto de encuentro?	✓	✓	✓	✓
q07	¿Qué proyectos tiene DNI?	✓	✓	✓	✓
q08	¿Cuánto cuesta el alquiler? (fuera ámbito)	✓	✓	✓	✓
q09	¿Cómo contacto con DNI?	✓	✓	✓	✓
q10	¿Qué es el refuerzo escolar?	✓	✓	✓	✗
q11	¿Puedo apuntarme si soy menor?	✗	✗	✓	✗
q12	¿Cuándo se fundó DNI?	✓	✗	✓	✓
Aciertos		10/12	8/12	11/12	9/12

Métricas de rendimiento (medias sobre las 12 preguntas)

Modelo	Servidor	Latencia media (s)	Tokens/s	Tokens salida
qwen2.5:3b	ollama_local	27,20	11,69	57,5
llama3.2:3b	ollama_local	24,96	10,47	54,1
gemma3:27b	poligpt	1,97	37,45	72,0
llama3.3:70b	poligpt	3,76	9,52	63,7

El comportamiento por tipo de pregunta fue el siguiente. En las preguntas factuales directas el sistema responde correctamente cuando la información está explícita en el corpus ("¿Qué es DNI?", "¿Cuántos voluntarios tiene DNI?" → +400 voluntarios activos). En logística funciona bien ("¿Cómo me apunto a los desayunos solidarios?", "¿Dónde es el punto de encuentro?" → Porta de la Mar de Valencia). Ante contradicciones entre documentos, los modelos más capaces (llama3.3:70b) presentan ambas versiones citando el archivo de cada una, mientras que qwen2.5:3b tiende a elegir una sin exponer la contradicción. La pregunta de síntesis "¿En qué se diferencian RESIS y COLES?" la fallan los 4 modelos: el retrieval recupera chunks de cada proyecto por separado pero ninguno contiene una comparación directa, por lo que el sistema responde correctamente que no tiene esa información (es un

límite del retrieval top-k, no una alucinación). Las preguntas fuera de ámbito se rechazan correctamente en todos los modelos.

5. Evaluación con RAGAs y métricas propias

Además del benchmark comparativo entre modelos, se evaluó la calidad del pipeline con el framework RAGAs sobre un subconjunto de cinco preguntas con respuesta de referencia (ground truth). RAGAs calcula cuatro métricas estándar a partir de la pregunta, la respuesta del sistema, los contextos recuperados y la referencia. Las medias obtenidas son:

Métrica RAGAs	Valor medio
Faithfulness	0,40
Answer relevancy	0,50
Context precision	0,50
Context recall	0,90

El valor alto de context recall (0,90) indica que la recuperación trae casi siempre el contexto que contiene la respuesta correcta, lo que es coherente con el buen comportamiento en preguntas factuales. Los valores moderados de context precision y faithfulness reflejan que, con $k=5$ y un modelo de generación de 3B parámetros, parte del contexto recuperado es ruido y el modelo introduce ocasionalmente matices no plenamente respaldados por las fuentes. La métrica answer relevancy se ve penalizada en las preguntas de rechazo correcto ("No tengo esa información en mis fuentes"), que RAGAs puntúa como poco relevantes pese a ser la respuesta deseada por el prompt anti-alucinación.

Se diseñaron además dos métricas propias orientadas al caso de uso de DNI. La **Source Citation Accuracy** verifica que la respuesta cite fuentes válidas del corpus; su valor medio es 1,00, es decir, todas las respuestas evaluadas atribuyen correctamente su procedencia. La **Contradiction Handling Score** mide si el sistema expone explícitamente las contradicciones entre documentos en lugar de elegir una versión en silencio; su valor medio es 0,10, un valor bajo que confirma de forma cuantitativa la limitación de los modelos pequeños descrita más abajo: solo en una de las preguntas el sistema verbalizó la procedencia de la información contradictoria.

6. Extras implementados

Frontend web (Streamlit)

Interfaz web que consume directamente el contrato consultar() sin reimplementar la lógica de dominio. Ofrece dos modos: un chat conversacional que muestra, por cada consulta, la respuesta, las fuentes citadas, los chunks recuperados con su score y las métricas de generación; y un modo de verificación de imágenes. Permite probar el sistema de forma visual y sirve de soporte para la defensa del proyecto.

Verificación de imágenes con AWS Rekognition

Permite subir una fotografía (un cartel, un horario impreso, una publicación) y comprobar si su contenido es coherente con la información oficial del corpus. El flujo extrae el texto de la imagen mediante el servicio detect_text de AWS Rekognition (OCR) y, a continuación, reutiliza el mismo retriever y generador del agente para contrastar ese texto con los

documentos oficiales, devolviendo un veredicto (COINCIDE, CONTRADICE o SIN INFORMACIÓN) acompañado de las fuentes. La integración es pertinente con el pipeline RAG: el resultado del OCR no se usa de forma aislada, sino como entrada de una verificación contra el corpus. Para mejorar la cobertura de la recuperación con texto procedente de OCR (más ruidoso que una pregunta redactada), en este flujo se amplió el número de chunks recuperados.

7. Análisis crítico, limitaciones y propuestas de mejora

Limitación 1 — Retrieval top-k no apto para síntesis comparativa

El sistema recupera los k chunks más similares semánticamente a la pregunta, pero no es capaz de agregar información dispersa entre múltiples documentos para construir una comparación. Una mejora sería implementar un retriever híbrido (semántico + BM25) o un paso de re-ranking que favorezca la diversidad de fuentes cuando la pregunta requiere síntesis.

Limitación 2 — Modelos pequeños locales no respetan el prompt de contradicción

qwen2.5:3b y llama3.2:3b ignoran la instrucción de presentar ambas versiones ante contradicciones y eligen una. Es una limitación de capacidad de seguimiento de instrucciones de los modelos de 3B parámetros. Con gemma3:27b o llama3.3:70b el comportamiento es correcto.

Limitación 3 — Latencia inaceptable en CPU pura

Con modelos locales en CPU, la latencia media es de 25-27 segundos por pregunta. Para un uso real esto es inviable. La solución es usar PoliGPT (1-5 segundos) o una GPU local.

Limitación 4 — Sin memoria de conversación

El sistema no recuerda preguntas anteriores. Cada consulta es independiente. El campo `conversation_id` está disponible en el contrato pero no se usa. Una mejora sería implementar un historial de conversación que se pase como contexto adicional.

Propuesta de mejora principal: implementar un retriever híbrido que combine similitud coseno (semántico) con BM25 (léxico) mediante rank fusion. Esto mejoraría especialmente las preguntas con términos muy específicos (nombres propios, horarios) donde el retrieval semántico recupera chunks temáticamente relacionados pero sin la información exacta.

8. Dificultades encontradas y cómo se resolvieron

Dificultad 1. El repo base estaba configurado para un proyecto diferente (GTI, no DNI). La colección ChromaDB se llamaba "gti_orienta", el corpus tenía archivos de GTI mezclados, y el prompt hablaba de estudiantes universitarios. Se corrigió añadiendo el .env completo y eliminando los archivos GTI del corpus.

Dificultad 2. El .env no se propagaba a SETTINGS porque la clase se inicializa en tiempo de import. Se resolvió cargando el .env manualmente mediante `os.environ.setdefault()` antes de cualquier import del paquete, en todos los scripts de entrada.

Dificultad 3. El endpoint de embeddings de Ollama cambió en la versión 0.24 de `/api/embeddings` (campo "prompt") a `/api/embed` (campo "input" y respuesta en

"embeddings" en vez de "embedding"). Se actualizó embedder.py para usar el nuevo endpoint y manejar ambos formatos de respuesta como fallback.

Dificultad 4. El chunker producía chunks de una sola línea (títulos de sección como "DINÁMICA DE TRABAJO") con alta similitud semántica con cualquier pregunta sobre el tema pero sin información útil. Se añadió un filtro que descarta chunks con menos de 80 caracteres tras el troceo.